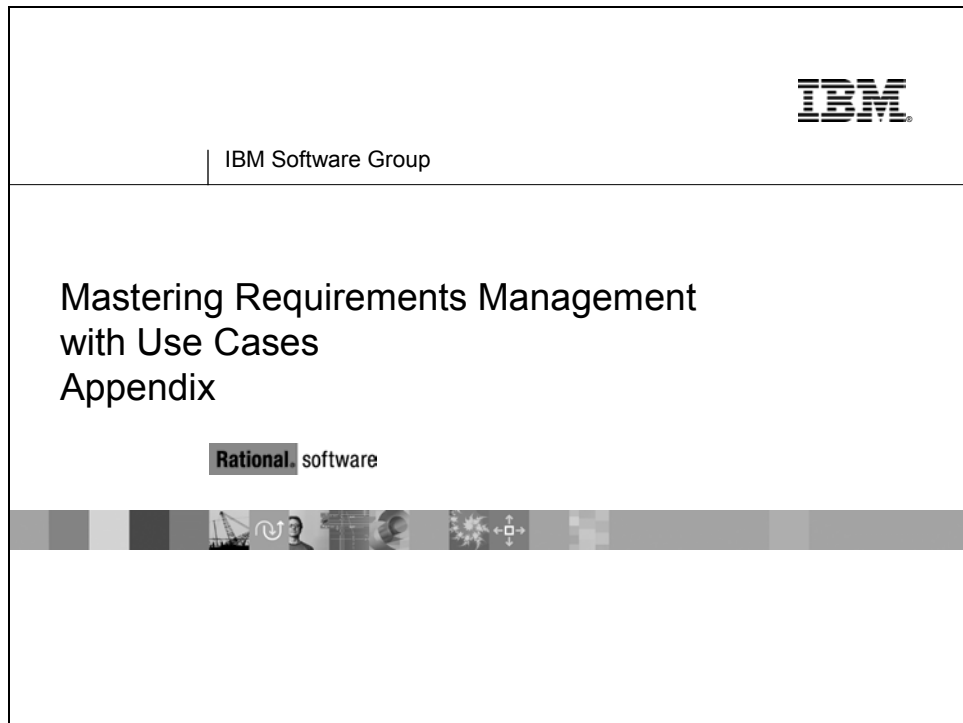




Appendix



Topics

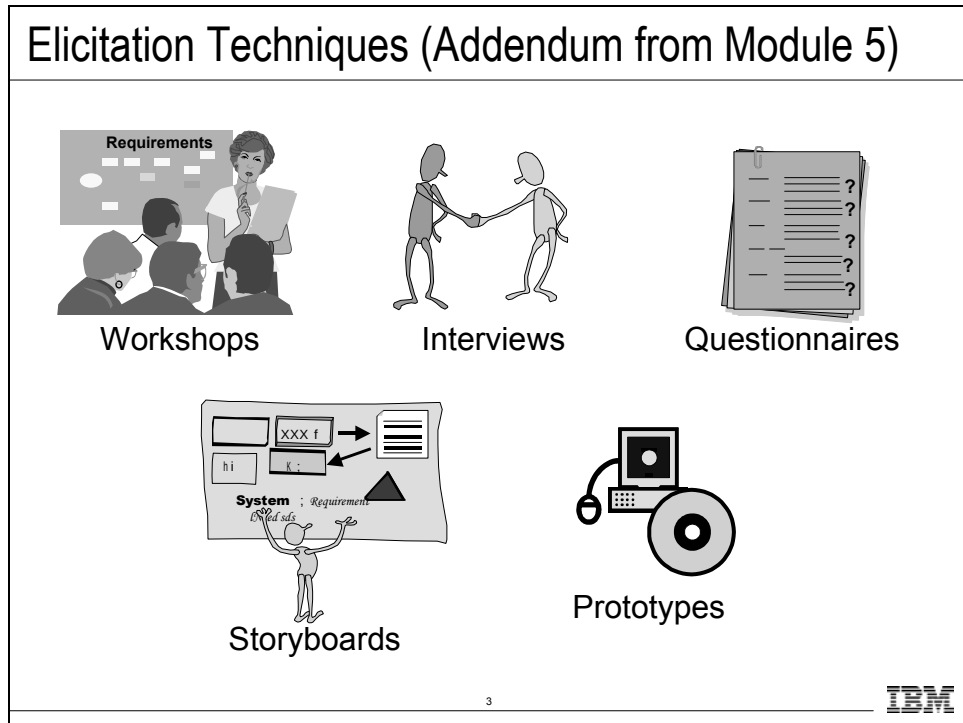
Elicitation Techniques.....	A-3
Use-Case Generalization	A-22
Requirements Across the Lifecycle	A-29

Contents

Contents

- ♦ Elicitation Techniques (Addendum to Module 5: Understand Stakeholder's Needs)
 - Workshops, Interviews, Questionnaires, Prototypes, and Storyboards
- ♦ Use-Case Generalization (Addendum to Module 10: Structure the Use-Case Model)
- ♦ Requirements Across the Lifecycle
 - Iterations and Phases
 - Requirements Reviews
 - Requirements Drive Development

Elicitation Techniques



Requirements Workshops

Requirements Workshops

- ♦ Create consensus on the scope, risks, and key features of the software system.
- ♦ Are directed by a facilitator.
 - Duration: three to five days
- ♦ Produce artifacts, such as:
 - Problem statement
 - Stakeholder Requests
 - Key features
 - Initial business object model
 - Use-case diagram
 - Prioritized risk list

4



Facilitation Process:

Problem Statement: Use a facilitation technique that brainstorms the Current State of the Business and then the Desired Future State of the Business. The gap between these two lists clearly identifies the areas of pain. For example, the problem that the business is grappling. In addition, it may help to further delve into the problem by building a Force Field Analysis of the Driving Forces that are helping the business drive to the desired Future State. Then identify the Restraining Forces that are holding the business from moving to the Future State. Use the Problem Statement template from the Vision document and build the Problem Statement as a group.

Key Features: Refer to student notes on brainstorming slide for details.

Business Object Model: The same process that was described in the Key Features is used here as well. The Brainstorming session is focused on the major nouns in the business. Instead of Step 4 in the brainstorming process, the relationships, multiplicity, and how the business uniquely identifies a single occurrence of the business class is explored.

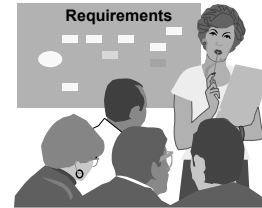
Use-Case Diagram: Refer to the student note on use-case workshop slide for details.

Prioritized Risk List: This process is identical to the Key Feature's process although you are brainstorming what the risks are to the project success and affinizing into groups that would all have the same mitigation strategy.

Requirements Workshop Benefits

Requirements Workshop Benefits

- ♦ Provides a framework for applying other elicitation techniques.
 - Use-case workshops, brainstorming, storyboarding
- ♦ Accelerates the elicitation process.
- ♦ Gathers all stakeholders together for an intense, focused period.
- ♦ Promotes participation by everyone.
 - Results available immediately



5

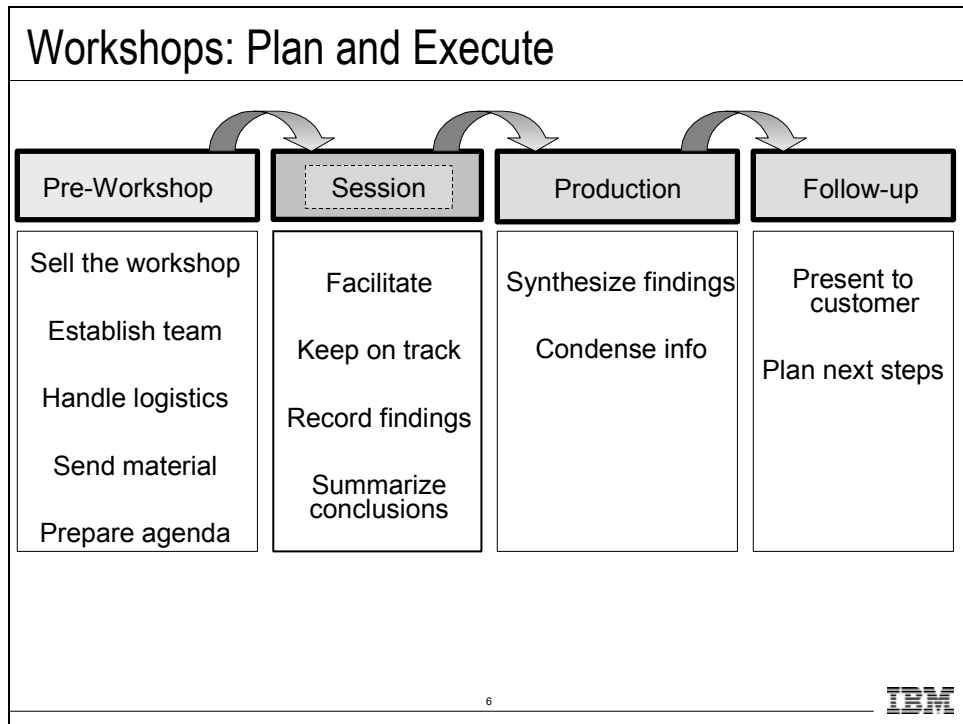
IBM

A requirements workshop is one of the most cost-effective and time-efficient means to get feedback. The same concepts are used in JAD (Joint Application Development) or RAD (Rapid Application Development) sessions. The basic idea is to get all the key players in the same room so that they can come up with a single set of requirements.

Interviews and other techniques for gathering requirements take a long time because each stakeholder has his or her own requirements, which may conflict with another stakeholder's requirements. Getting everybody together accelerates requirements gathering because the conflicting requirements are discovered and worked through in a single workshop session.

Requirements workshops are often combined with other group-oriented techniques. For example, if the participants in the requirements workshop are having trouble coming up with ideas, brainstorming might be useful. For another example, a requirements workshop is an ideal setting for developing the initial use-case model.

Workshops: Plan and Execute

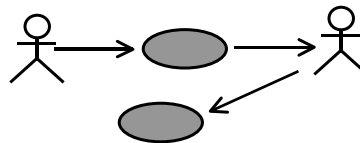


Here are some suggestions on planning and executing a requirement workshop. Make sure you gather the right stakeholders. Who are they?

How Do Use Cases Help Elicit Needs?

How Do Use Cases Help Elicit Needs?

- ♦ Provide stakeholders' perspectives.
- ♦ Promote discussion.
 - Why is the system needed?
 - Who/what interact with the system (actors)?
 - What do users want to achieve by using the system (use cases)?
 - What interfaces should the system have?



Use-Case Model

7



Uses cases are ideal for discussing requirements with stakeholders because use cases express the user perspective.

What do the users want to accomplish with the help of the system?

Use cases and actors give context for finding the needs of the stakeholders.

Who needs the system, the ways they want to use the system, and their interfaces to the system.

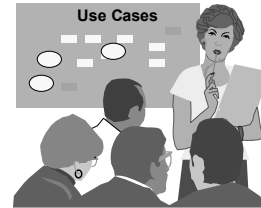
In conjunction with identifying use cases, you might need to use other elicitation techniques. For example, it is common to hold a requirements workshop for the purpose of identifying use cases. Storyboarding is a technique for exploring the sequence of actions that are part of the use case.

Use-case modeling is an extremely important elicitation technique. The course devotes much of the following module to it.

A Use-Case Workshop

A Use-Case Workshop

- ♦ Is organized as a requirements workshop.
- ♦ Includes small groups (less than 10 people).
 - Half development group
 - Half user representatives
- ♦ Produces an initial model of the system quickly.
- ♦ Uses common tools:
 - Easels and flip charts
 - Sticky pads and colored pens
 - Walls and whiteboards



8

IBM

Use-case workshops are usually run as part of a requirements workshop. The goal is to rapidly establish clear boundaries of scope for the system. The group will identify actors, use cases, and domain experts for each use case identified. It needs to be well understood that the use-case model will be refined throughout the project, but the core use cases usually remain stable. The group achieves consensus on the names of the use cases and their description, which forms an important foundation for establishing a common vocabulary.

Brainstorm all the actors. You always get some that are not real actors, but the process filters those. Make sure you understand who are the major actors that are initiating activity in the system. This may be accomplished by a quick bullet vote, but often the group can not reach consensus that quickly.

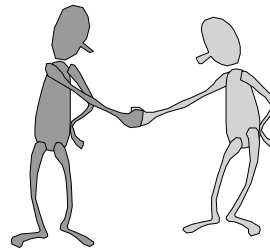
Starting with the primary actors and identify each use case that they initiate. Add any other actors that may interact with that use case. The use-case model will not be perfect. Err on the side of more use cases than less because they can always be collapsed later. You do not want to lose any important information.

After the major use cases have been identified the garbage actors just go away. Go through the list of use cases one more time to capture a sentence or two about what the goal of the use case is and who are the domain experts that should be interviewed to get all the detailed information. **This is an important step. Too often when this is skipped, people complain that the right folks were not asked. This step prevents a lot of future problems.**

Interviews

Interviews

- ♦ Provide a simple and direct technique.
- ♦ Gain understanding of problems and solutions.
- ♦ Consider all perspectives.
 - Users
 - Customers
 - Other stakeholders



9

IBM

Interviewing is probably the most useful technique to directly get needs from your stakeholders. It is helpful to interview key stakeholders for your project.

Interviewing is a direct technique that can be used in problem analysis and requirements elicitation. Interviews are designed to gain an understanding of problems and potential solutions from the perspectives of the users, customers, and stakeholders.

Interview Tips

Interview Tips

- ♦ Avoid asking people to describe things they don't usually describe.
 - Example: Describe how to tie your shoes.
- ♦ Avoid "Why...?" questions.
- ♦ Ask open-ended questions.
- ♦ Listen, listen, listen!

10



Here are some hints to help you avoid the dangers of unproductive interviews.

- Do not ask people to describe things they do not usually describe. This assumes that users can describe complex activities. In general, people can do many things they cannot describe. For example, how could you describe tying your shoes?
- Avoid asking "why" questions that are directed at the person you are interviewing. For example, "Why do you do it that way?" may cause a defensive response. However, "Why are you required to deliver this in one business day?" would not.
- Ask open-ended questions and be patient while waiting for the answers. Do not expect simple answers. Allow interviewees to complete their train of thought and allow them time to think. Do not cut them off, but do ask follow-up questions.

Context-Free Questions

Context-Free Questions

- ◆ Are high-level, abstract questions.
- ◆ Explore needs from stakeholder perspective.
 - User's problems.
 - Potential solutions.
- ◆ Are always appropriate.
 - Unbiased with application or solutions knowledge.
- ◆ Are typically posed early in a project.

Gause & Weinberg, 1989

11



The context-free question is a high-level, abstract question that can be posed early in a project to obtain information about global properties of the user's problem and potential solutions. It helps you gather information to understand the real problem without biasing the user's answers.

Four types of context-free interview questions are considered:

- User
- Process
- Product
- Meta-questions

The context-free question is discussed in the book *Exploring Requirements: Quality Before Design* by Gause & Weinberg. This book is listed as recommended reading in the appendix.

Types of Context-Free Questions

Types of Context-Free Questions

- ◆ User questions
 - Who are the users?
 - What the key responsibilities of each user?
 - What is the user's background, capabilities, environment?
- ◆ Process questions
 - What is the problem?
 - How do you currently solve the problem?
 - How would you like to solve the problem?
 - Where else can the solution to this problem be found?

Gause & Weinberg, 1989

12



Types of Context-Free Questions (cont.)

Types of Context-Free Questions (cont.)

- ♦ Product questions
 - What environment will the product encounter?
 - What business problems could this product create?
 - What are your expectations for usability? reliability?
- ♦ Meta-questions
 - Do my questions seem relevant?
 - Are you the right person to answer these questions?
 - Are your answers requirements?
 - Is there anything else I should be asking you?

Gause & Weinberg, 1989

13



Non-Context-Free Examples

Non-Context-Free Examples

- ♦ Leading questions
 - You need a larger screen, don't you?
- ♦ Self answering questions
 - Are fifty items about right?
- ♦ Controlling statements
 - Can we get back to my questions?
- ♦ Too long and too complex
 - I have a three part question, ...

What are better questions to ask?

14



It is easy to conduct an interview that does not yield useful information if context-free questions are not used.

Questionnaires

Questionnaires

- ♦ Give access to a wide audience.
- ♦ Appear scientific because of statistical analysis.
- ♦ Apply to broad markets where questions are well-defined.
- ♦ Are powerful, but not a substitute for an interview.
- ♦ Assumptions:
 - Relevant questions can be decided in advance.
 - Questions phrased so reader hears as intended.

© 1994 by Alan M. Davis

15



A questionnaire can be useful for reaching a large audience to gather data, but it should often be prefaced by interviews to determine the correct questions to ask. How many customers should be interviewed to determine a good cross-section of questions? Usually 12 to 15 people is adequate.

A questionnaire is especially useful for gathering data if the issues are well-understood and you just need to get a statistical analysis of the broad market. For example, Rational University uses an evaluation questionnaire to gather feedback on its courses. Most items are categorized ("On a scale of 1 to 5 ...") to allow for statistical analysis.

Always leave some open-ended questions to allow for new ideas. You may also want to follow up on the results by interviewing some of the participants to understand their reasoning in more depth than you can get from a questionnaire.

Questionnaires can be very difficult to get right. A lot of time needs to be spent designing and testing the questionnaire. If you do not do this, then quite often you find the answers are of little or no value to you in helping solve your problem.

Statistical analysis lends an air of scientific truth to questionnaire results. However, you need to avoid biasing the results: send it to a representative sample of potential stakeholders and ask the questions in a way that does not bias the answers.

Have you ever used a questionnaire for gathering requirements? Have you had a situation where it may have been useful technique for eliciting requirements?

Observation/Contextual Interviews

Observation/Contextual Interviews

- ♦ Definition
 - Observe real user in the field.
 - Master-apprentice model: User explains his activities and answers questions.
 - Use real data in real situations.
- ♦ Advantages
 - Deliver requirements from situated activities.
 - Probe inherent/tacit knowledge.
- ♦ Issues
 - Experienced interviewer needs to keep focused on interview goals.
 - Expensive and time-consuming to analyse data.
 - Not always applicable to do in real situations.

Beyer & Holtzblatt, 1997

16



Contextual interviews as described in [Beyer and Holtzblatt, 1997] apply four main principles:

•**Context:** The principle of context expresses that the interview must not take place in an artificial or simulated work environment but always have to take place at the interviewee's real workplace to see the his actual work as it unfolds. Further, contextual interviews capture *ongoing experience* rather than *summary experience*, and *concrete data* rather than *abstract data*. This means that the interview has to kept concrete by exploring real artefacts and real events from the interviewee ongoing work and not talk about it in the general.

•**Partnership:** The objective of the partnership principle is to make interviewees collaborators in understanding and ultimately improving his work (*the shared quest*). It is important that he becomes convinced that this partnership benefits him the most.

•**Interpretation:** Whereas the principle of context strives for keeping the interviewee to stay focused on his concrete work, the analyst ultimately has the task to create abstractions from his observations through interpretation. Interpretation assigns a meaning to the observations and aims to determine what the interviewee's words and actions mean together. Confronting the interviewee with interpretations provides a structure for his work and helps him to validate them.

•**Focus:** The fourth principle expresses that a clear focus has to steer the conversation, i.e., that the interview should stick to a specific objective or agenda for the interview and should not get stuck in details which serve no purpose to this objective.

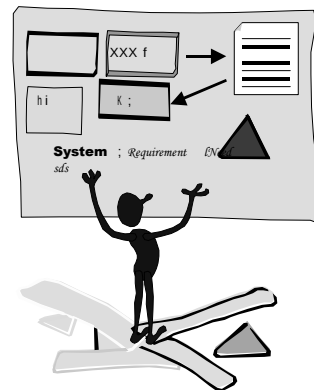
A contextual interview, which should normally take two hours in length.

Storyboards

Storyboards

- ♦ User interface *paper prototype*.
- ♦ Visually tell and show:
 - Who/what the players are (actors).
 - What happens to them.
 - When it happens.

*Movies, cartoons, and
animated features all
begin with storyboards.*



17

IBM

Storyboarding gets people involved in developing the *story* for a sequence of actions.

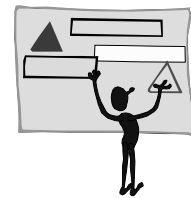
Storyboarding comes from the movie industry but has been adapted for the computer industry.

Storyboarding is often a good way to get the users talking about the sequence of actions that become use cases or the steps in a use case.

Storyboard Benefits

Storyboard Benefits

- ♦ Gathers and refines customer requirements in a user-friendly way.
- ♦ Encourages creative and innovative solutions.
- ♦ Encourages team review.
- ♦ Prevents features that no one wants.
- ♦ Implements features in an accessible and intuitive way.
- ♦ Eases the interview process.
- ♦ Avoids blank-page syndrome.



Shurtleff '94

18

IBM

Storyboarding is useful for eliciting needs, high-level requirements, and detailed requirements. It is a way to overcome the “blank page” syndrome when nobody has any ideas.

Prototypes

Prototypes

- ♦ Demonstrate some or all of the externally observable behaviors of a system.
- ♦ Are used to:
 - Demonstrate understanding of the problem domain.
 - Gain feedback on proposed solution.
 - Validate known requirements.
 - Discover unknown requirements.
 - Create simulations.

19



In RUP®, the user-interface designer leads and coordinates the prototyping and design of the user interface by:

- Capturing requirements on the user interface, including usability requirements.
- Building user-interface prototypes.
- Involving other stakeholders of the user interface, such as the users, in usability reviews and usability testing sessions.
- Reviewing and providing the appropriate feedback on the final implementation of the user interface (as created by other developers, designers, and implementers).

The following people use the user-interface prototype:

- Use-case authors to understand the user interface for a use case.
- Object analysts to understand how the user interface influences the analysis of the system.
- Designers to understand how the user interface affects the system and what the user interface requires from the *inside* of the system.
- Those who test the classes to plan testing activities.

Why Use Prototypes?

Why Use Prototypes?

- ◆ Elicit and understand requirements.
- ◆ Prove and understand technology.
- ◆ Reduce risk.
- ◆ Enhance shared understanding.
- ◆ Improve
 - Cost and schedule estimates
 - Feature definition

20



Prototypes are built early, either during Inception or in the beginning of the Elaboration phase, and before the whole system (including its *real* user interface) is analyzed, designed, and implemented.

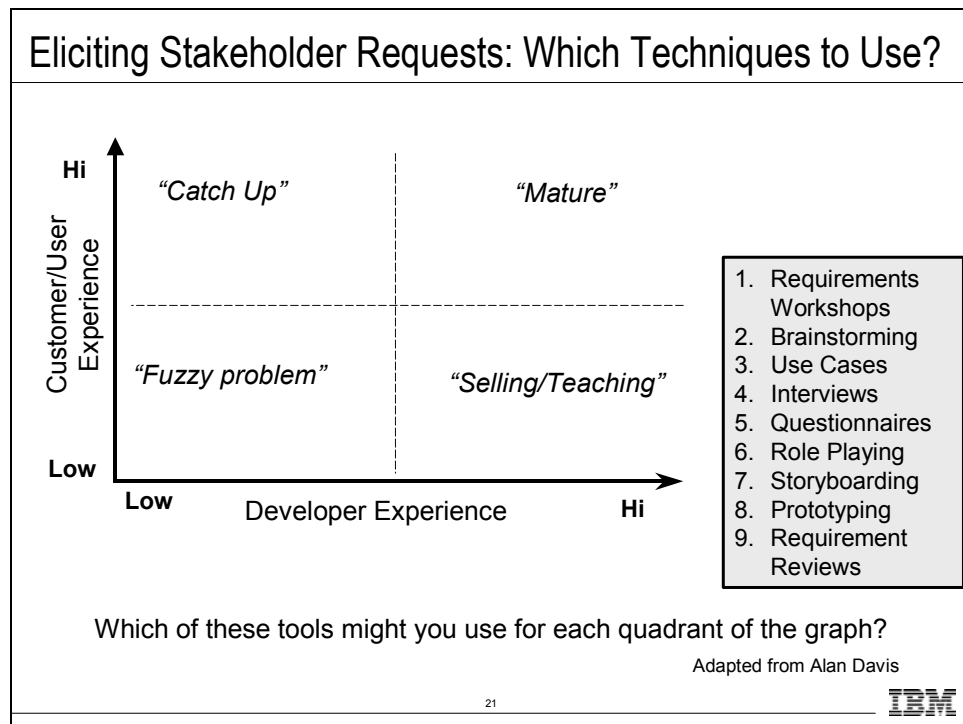
Note: The main purpose of creating a prototype is to be able to expose and test both the functionality and the usability of the system before the real design and development starts. This way, you can ensure that you are building the right system before you spend too much time and resources on development. Often important features are found to be of little value or vice versa.

To achieve this early testing, it follows that the prototype should be significantly cheaper to develop than the real system, while still having enough capabilities to be able to support a meaningful use test.

A project does not need to consider all aspects in the prototype. Instead, it can often be appropriate to leave some aspects to the implementers and thus not deal with them in the prototype.

While you design the prototype, you should continuously implement your design and expose it to others. Consider any project-specific User-Interface Guidelines.

Eliciting Stakeholder Requests: Which Techniques to Use?



Which of these tools might you use for each quadrant of the graph?

The experience level of the customer or user and the experience level of the development team can often be critical factors in determining which techniques you apply in your project. For example, if the customer has a lot of experience in a particular domain but the developer has little experience, then the developer must "Catch Up" with the customer. What techniques would be particularly good to help an inexperienced developer to catch up with the very knowledgeable customer?

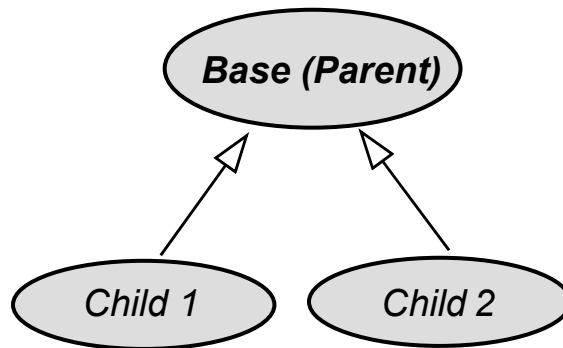
The selection of elicitation techniques can also be related to experience in different dimensions:

- The problem domain
- The development methodology, such as object oriented
- The tools being used and so on

You may, for example, start off in "Fuzzy Problem" and make use of techniques 1, 2, and 7. After you have performed these elicitation techniques, you may move into another quadrant and then additional techniques that are more appropriate.

Use-Case Generalization

Use-Case Generalization (Addendum from Module 10)



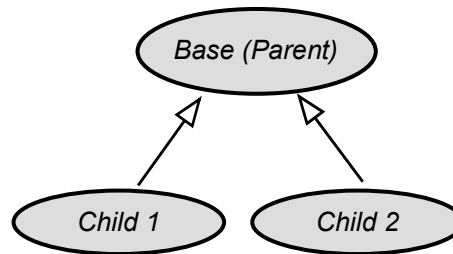
22

IBM

What Is Use-Case Generalization?

What Is Use-Case Generalization?

- ♦ A relationship from a child to a parent use case.
 - Describes general shared behavior in parent.
 - Describes specialized behavior in children.
 - Shares a common goal.



23

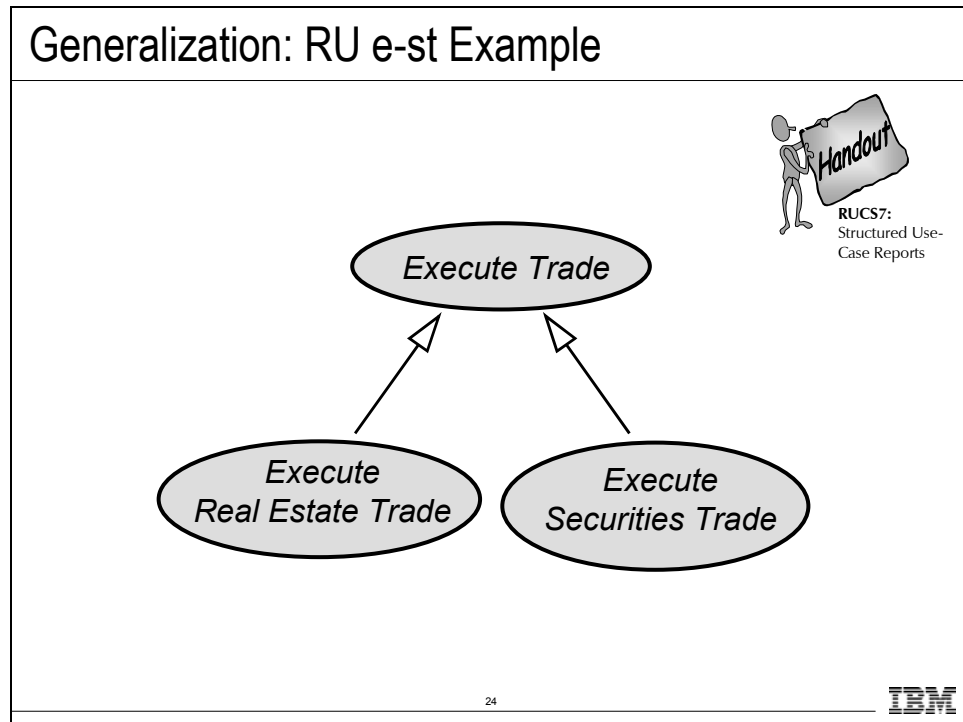
IBM

Use-case generalization is a rarely used form of structuring. A parent use case may be specialized into one or more child use cases that represent more specific forms of the parent. Neither parent nor child is necessarily abstract, although the parent in most cases is abstract. A child inherits all structure, behavior, and relationships of the parent. Children of the same parent are all specializations of the parent. This is generalization as applicable to use cases.

You only use generalization if the children and parent share similar structure and purpose. If the goal of the child is a not specialized or a refined goal if the parent, then the generalization is not applicable.

Note the direction (and hollow arrowhead) of the generalization arrow.

Generalization: RU e-st Example



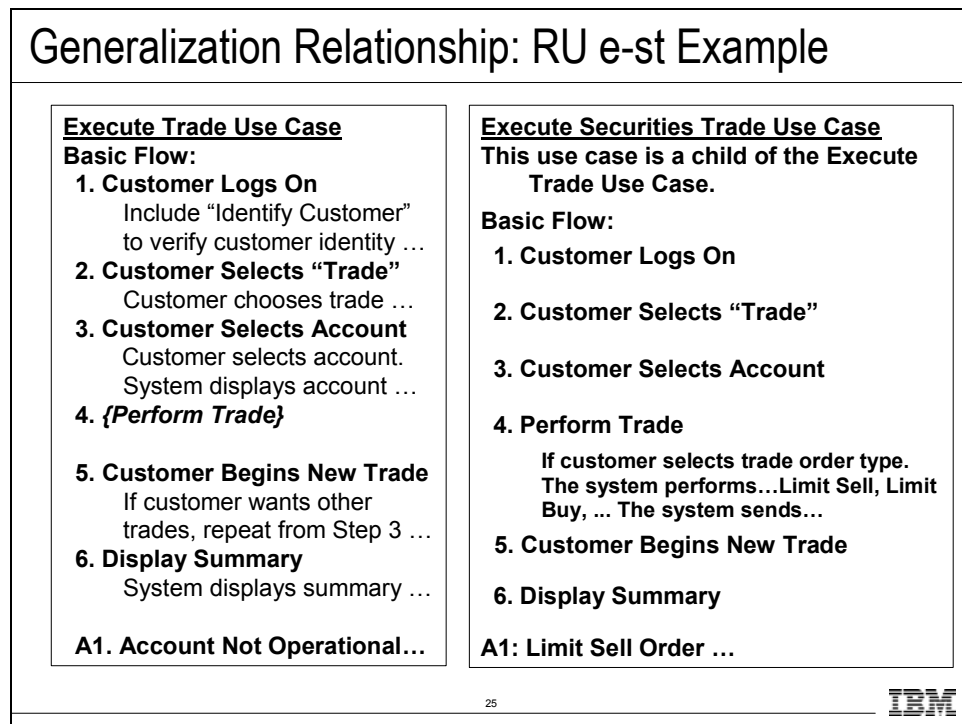
In the example from the RU e-st system, you have added to the functionality of the system. The simpler, former version of the RU e-st system only performed securities trading (stocks and mutual funds). Now you want to allow other kinds of trading: real estate (houses and vacant land) and maybe other kinds of trading (agricultural products, gold, and so on). Each of these other kinds of trading involves very different procedures. So, you have a family of use cases, each performing a different kind of trade.

The parent use case is Execute Trade, which now contains only the steps in it that are common to all of the children use cases. Each child use case uses the common steps in the parent use case, and only contains the detailed steps necessary for its own specialized kind of trading. The child use cases are Execute Securities Trade, Execute Real Estate Trade, and others not shown, such as Execute Agricultural Trade.

The example of generalization is based on an actual system that was developed for a bank in Australia. They have a computerized system that lets companies manage their accounts. The software lets them create a sequence of banking instructions offline and then send the instructions to the bank when the system dials in. The types of usage are: domestic funds transfer, international funds transfer, loan payments, payroll, and others. The usages could all be alternative flows in a single use case, Manage Accounts.

The trouble is that managing each type of account has a lot of special behavior, so the entire use case would be about 200 pages. The use-case specifier chose to have separate use cases related by generalization because all the types of usage share common purpose and behavior.

Generalization Relationship: RU e-st Example



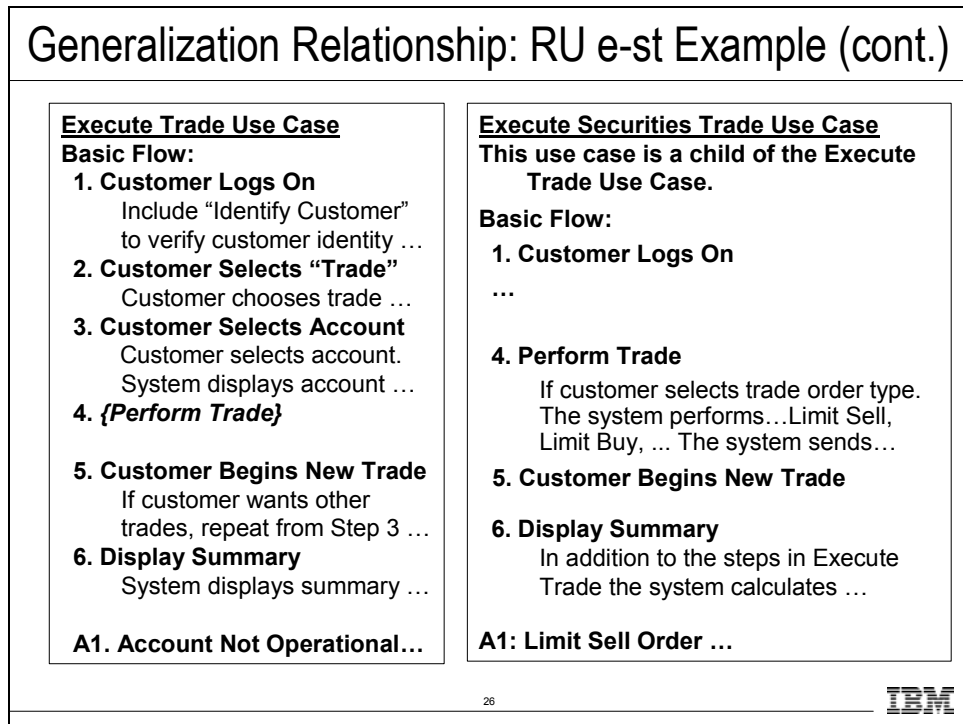
The example shows one way of describing use-case generalization in the textual description of the parent and child use cases. On the left of this example is the parent use case, Execute Trade. The flow of events in Execute Trade contains all the behavior common to its children. In this example, the only step in the basic flow that is not common to all the children is Step 4, Perform Trade. Step 4 is left blank and its label is in italics to indicate that the children use cases must fill in the detail for performing that step. Italics font is used in the parent use case because it is the UML convention for identifying something that is abstract.

On the right of this example is one child use case, Execute Securities Trade. The labels for Steps 1, 2, 3, 5, and 6 in the basic flow are given but the details are left blank. The detailed behavior for these steps is filled in from the corresponding steps of the parent use case. The details for Step 4, Perform Trade, are specified in the child use case because the procedure in Step 4 is specific to performing securities trading. In addition, the child use case has many alternative flows that are specific to securities trading.

Note that there are other ways to relate these use cases. For example, the Execute Securities Trade use case could «include» behavior from the Execute Trade use case. As another possibility, Execute Securities Trade use case could «extend» the Execute Trade use case, for the optional behavior of trading securities. The reason to use generalization is that generalization emphasizes that this really is a family of use cases: they share the same structure, behavior, and intent.

It is very rare to use use-case generalization. It can often obscure rather than clarify the use-case model. Rational recommends that you use it very sparingly, if at all.

Generalization Relationship: RU e-st Example (cont.)

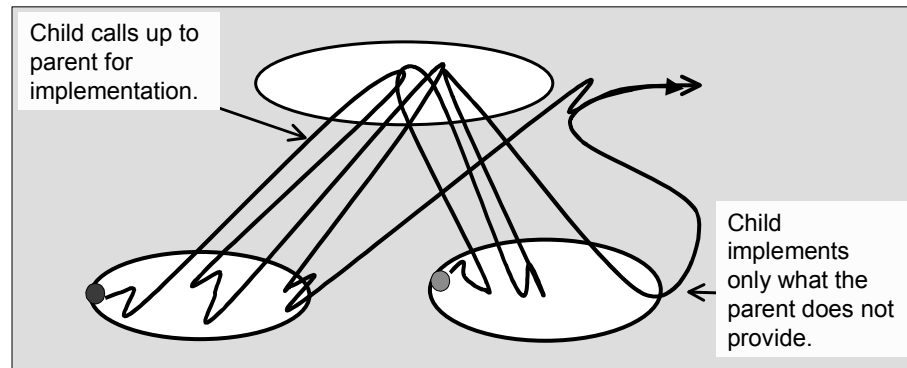


Use-case generalization can also be used to specialize non-abstract steps.

Execution of a Use-Case Generalization

Execution of a Use-Case Generalization

- ♦ A use-case instance executing a child use case follows the flow of events described for the parent use case, inserting additional behavior and modifying behavior as defined in the flow of events of the child use case.



A use-case instance executing a child use case follows the flow of events described for the parent use case by inserting additional behavior and modifying behavior as defined in the flow of events of the child use case.

In general, you do not have to describe the generalization relationship itself. Instead, in the flow of events of the child use case, you specify how new steps are inserted into the inherited behavior and how inherited behavior is modified.

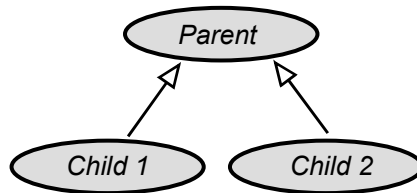
Extension and generalization are sometimes confused. The key distinction between extension and generalization is that extension changes the use case into something different (the goal can change), whereas generalization keeps it the same (the goal is preserved between the parent and the child).

Note: If the child is specializing more than one parent (multiple inheritance), you must explicitly state how the behavior sequences from the parents in the specification of the child. However, use of multiple inheritance, while technically possible, would make your requirements almost impossible to comprehend and should be avoided at all costs.

Why Use Use-Case Generalization?

Why Use Use-Case Generalization?

- ♦ Shows common behavior, structure, and purpose in two or more use cases.
 - Show that children are part of a use case family.
 - Avoid describing same behavior multiple times.
 - Assure common behavior remains consistent.
 - Allow reuse of requirements.



28

IBM

Generalization is used when you find two or more use cases that have commonalities in behavior, structure, and purpose. In such a case, you can describe the shared parts in a new, often abstract, use case that is then specialized by child use cases. The parent use case may consist of both behavior and characteristics that are shared.

Experience in the field suggests that generalization is rarely used in actual practice. Instead, the specialized behavior is usually modeled with alternative flows inside the base use case.

Requirements Across the Lifecycle

Requirements Across the Lifecycle

- ♦ Use-case models at the end of each phase of the lifecycle.
- ♦ Iteration assessment.
- ♦ Types of requirements reviews.
- ♦ Requirements drive *Analysis and Design*.
- ♦ Requirements drive *Test*.

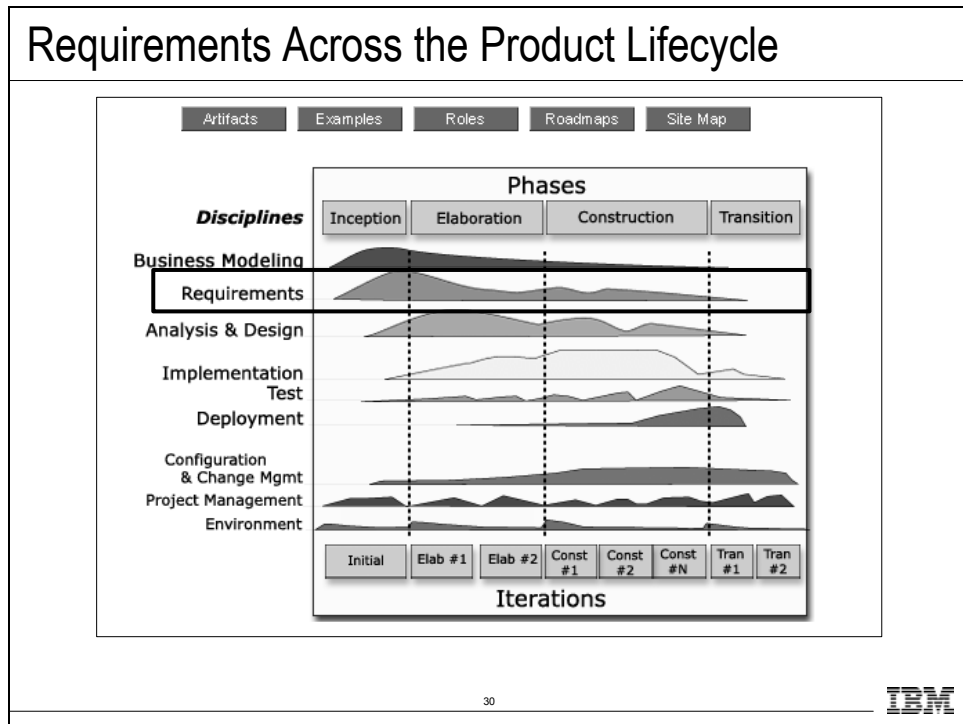
29



In this module, you will look at how requirements evolve across the product lifecycle.

The iterative process allows us to refine the model in smaller increments at each iteration and also helps us maintain control of the changes in requirements and use needs throughout the product lifecycle.

Requirements Across the Product Lifecycle



The nature of the work performed in the iterations varies greatly across the lifecycle. Early iterations focus on requirements capture and architectural design; in construction, iterations tend to focus on design, implementation, and test.

Also, depending on where you are in the lifecycle, the way you perform requirements capture within an iteration may vary.

Each iteration in Rational Unified Process® involves one pass through the activities in each of the core disciplines, including requirements.

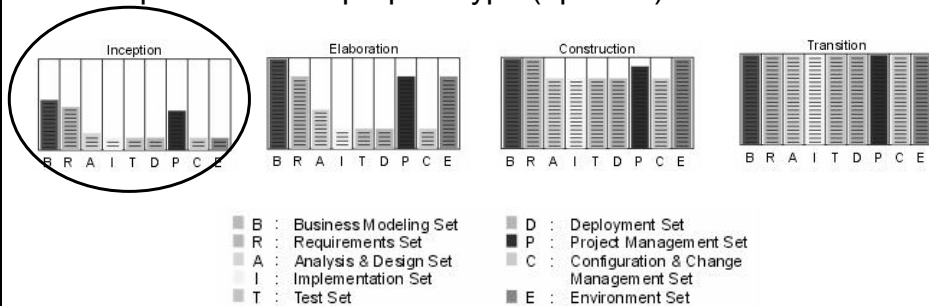
Look at the typical state of our requirements at the end of each of the phases of our iterative process.

Where is this done in your own software development process?

Inception Iterations: Typical Requirements Results

Inception Iterations: Typical Requirements Results

- ♦ Collect information to develop the vision and business case:
 - A draft of a survey of the use-case model
 - An initial vocabulary
 - A few use-case flows of events (requirements capture)
 - A “proof of concept” prototype (optional)



31

IBM

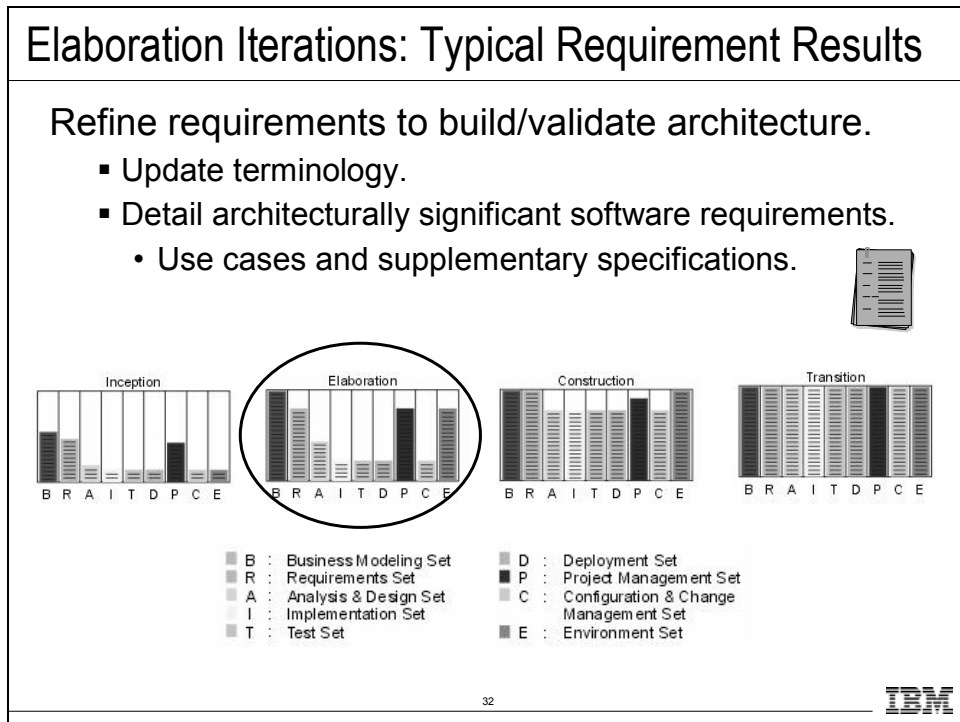
The purpose of the inception phase is to establish the business case for the system and specify the project scope. To accomplish this, you must understand the context of the system, the external entities with which the system interacts, and the nature of that interaction at a high level.

In the inception phase, the goal of an iteration is to gather enough information to develop the business case.

In a typical example, only one preliminary iteration is performed in the inception phase. No design, implementation, or testing follow on the requirements capture. Typical activities include having the team:

- Perform a thorough analysis of the problem domain to help define the problem to be solved and what it is worth to the business to solve.
- Conduct a requirements workshop where they discuss some core concepts in their vocabulary, identify an outline of actors and use cases, and briefly describe the actors and use cases.
- Hold an informal review with the project stakeholders to make sure they are on the right track and that there is a business case for the project.
- Decide on how to take care of all change requests.
- Develop a “proof of concept” prototype to help obtain project funding.

Elaboration Iterations: Typical Requirement Results



The purpose of the elaboration phase is to establish a sound architectural foundation, develop the project plan, and eliminate the highest risk elements of the project. To accomplish these objectives, you must have the “mile wide and inch deep” view of the system. Architectural decisions have to be made with an understanding of the whole system: the scope, major functionality, and nonfunctional requirements, such as performance requirements.

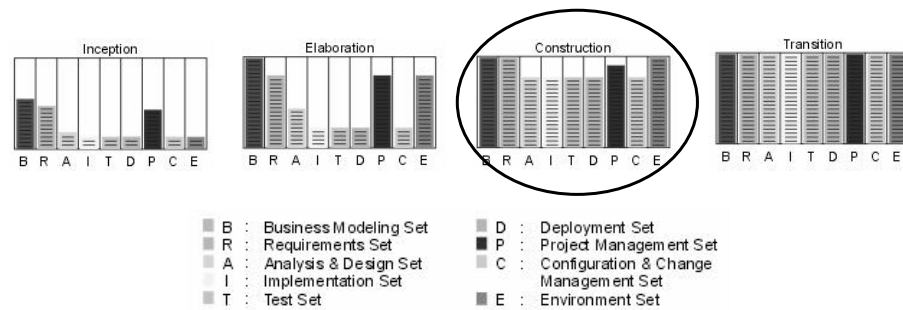
In the elaboration phase, the goals are to refine the architecturally significant requirements and to build and validate an architecture. Iteration activities include:

- Prioritize architecturally significant use-case scenarios to include in this iteration. These decisions are based on a first understanding of the architecture of the system and what use-case scenarios would be critical for that architecture.
- Write the flow of events for all the prioritized scenarios. Review sessions with domain experts are performed.
- Manage change that occurs as the requirements are refined and implemented.

Construction Iterations: Typical Requirement Results

Construction Iterations: Typical Requirement Results

- ◆ Build the complete system.
 - Change requests on use-case's flow of events.
 - Updated use-case flow of events.
 - Emphasis getting the bulk of the work done.



33

IBM

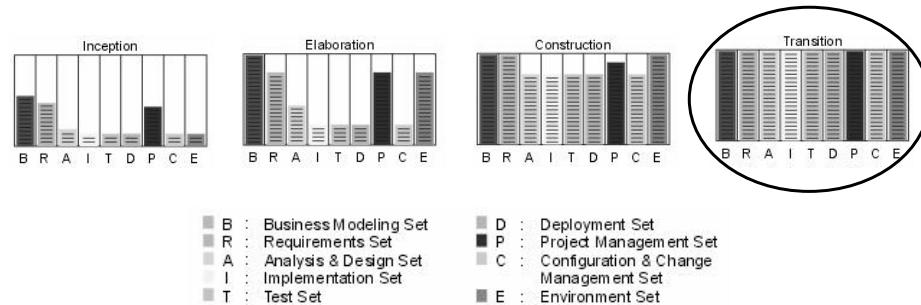
The purpose of the construction phase is to get the mass of the work done. The major risks to the project have been eliminated. The goal is to iteratively develop a complete software product that is ready for transition into the user community, starting from a baselined architecture. Thus, the remaining use-case scenarios must be completed during this phase, as well as, all remaining software implementation and test.

In the construction phase, the architecture and use-case scenarios implemented in Elaboration should be stable. The emphasis is on completing the use-case descriptions, design, implementation, and test of the remaining use-case scenarios.

Transition Iterations: Typical Requirements Results

Transition Iterations: Typical Requirements Results

- ◆ Requirements should not change.
- ◆ Similar to typical construction-phase iteration if new features are added.



34

IBM

The purpose of the transition phase is to transition the software product into the user community. Once the product has been given to the end user, issues usually arise that require you to develop new releases, correct problems, or finish some of the features that may have been postponed.

Requirements should not change at this late stage of the project.

In the transition phase, the activities performed during an iteration depend on the goal (fixing bugs, implementation, test, and transitioning data from any existing systems are usually enough).

Fixing bugs may involve fixing a problem discovered in the requirements. If new features have to be added, the iteration is similar to the construction phase.

Iteration Assessment

Iteration Assessment
◆ Apply evaluation criteria. ▪ Functionality. ▪ Performance. ▪ Capacity. ▪ Quality measures. ◆ Consider external changes. ▪ Examples: changes to requirements, user needs, competitor's plans. ◆ Determine what rework, if any, is required. ◆ Assign rework to remaining iterations. ◆ Assess project schedule.


Each iteration is concluded by an iteration assessment where the development organization pauses and reflects on what has happened, what was achieved or not (and why), and it uses the lessons learned to improve the organization and the process for future iterations.

For this iteration assessment, you assess the results of the iteration relative to the evaluation criteria that was established in the iteration plan: functionality, performance, capacity, and quality measures.

This is a critical step in making the iterative approach successful. If this is not done properly, the benefits of the iterative approach will be lessened.

Also, remember that at times the right thing to do in this step is to revise the evaluation criteria rather than reworking the system. Sometimes, the benefit of the iteration is in revealing that a particular requirement is not important, is too expensive to implement, or creates an architecture that cannot be maintained. In these cases, a cost/benefit capture must be done. This is a business decision.

Reviewing Requirements

Reviewing Requirements



- ♦ **Informal reviews**
 - Find errors.
 - Schedule whenever needed.
 - Include small team, possibly including QA.
- ♦ **Formal reviews**
 - Decide whether to proceed to next phase.
 - Schedule during milestones and tollgates.
 - Include large review team, including customers.

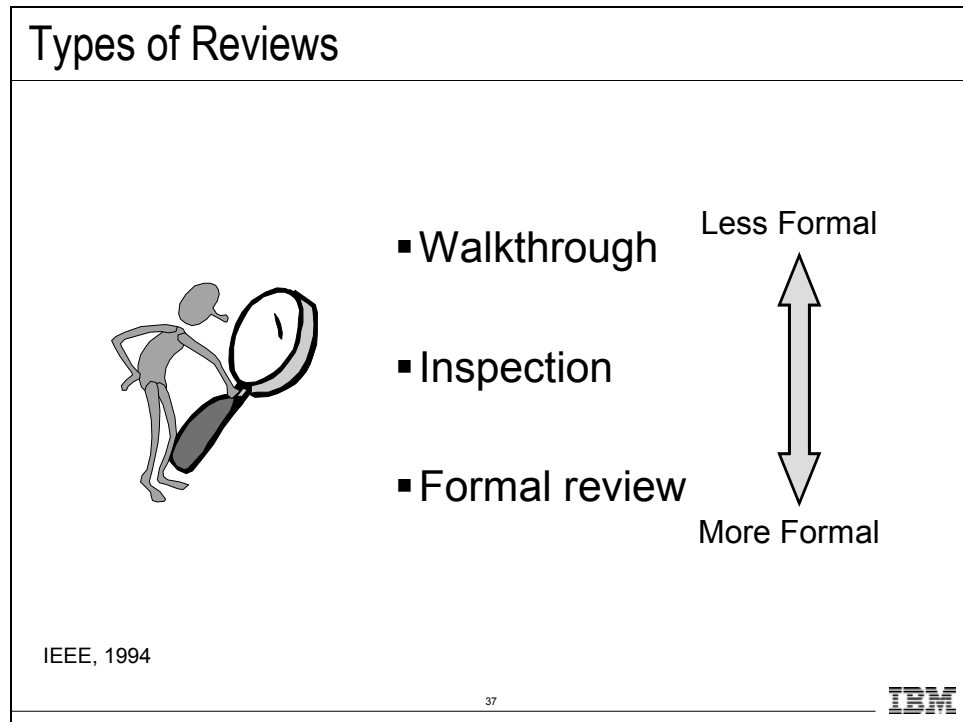
36



All new and changed requirements (use cases) need to be reviewed by a review group.

There are both formal and informal reviews.

Types of Reviews



As you progress through the development lifecycle, the reviews become more and more formal until the final formal review is accepted.

IEEE defines three types of reviews of different levels of formality and ceremony.

Reviewing use cases is usually done at the scenario level. By reviewing entire scenarios, the reviewers read a complete story and are likely to conduct better reviews.

Review Requirements: Walkthrough

Review Requirements: Walkthrough

- ♦ Purpose
 - Find errors in an early stage.
 - Find deviations from approved style, technique, standards.
 - Informing.
- ♦ Participants
 - A few project members, need not be prepared.
- ♦ Procedure
 - Analyst.
 - Gives an overview of the results.
 - Walks through reviewed chapters, other participants comment.
 - Documents errors found.

38



Walkthroughs are the least formal.

This was discussed during the elicitation techniques section when doing a walkthrough of a customer-generated specification.

Review Requirements: Inspection

Review Requirements: Inspection

- ♦ Purpose
 - Share views from different parts of the organization.
 - Find errors and problems early.
 - Approve, revise, or reject decisions.
- ♦ Participants
 - Moderator, recorder, author, and inspectors.
- ♦ Procedure
 - Organize before the meeting.
 - Keep meeting focused (moderator).
 - Record all issues (recorder).
 - Look for and discuss errors (inspectors).

39



Inspections are more formal than walkthroughs and are used to gain information as well as to find problems or errors.

Inspectors can be:

- Users of the system and managers of the users
 - Representatives of all departments using new system
 - Not just those who are involved in this use case
- Process owners
- Domain experts
- Designers of the system

The role that the different groups play during the inspection depends on who wrote the use case (or other requirements document). If it was written by a specific analysis team familiar with the problem domain, then the designers are an important inspector group. On the other hand, if the designers wrote the use cases, then it would be very important to include the inspectors with the domain expertise.

Advance preparation is crucial to a successful inspection. At least one week before the meeting: invite Inspectors (**Hint:** less than 8) and distribute materials, instructions, and questions. Rational recommends no more than 50 pages in one inspection. Have inspectors read materials and write comments before the meeting.

Why is it important that inspectors write their comments ahead of time?

Review Requirements: Formal Review

Review Requirements: Formal Review

- ◆ Purpose
 - Ensure that results are complete and consistent.
 - Decide on continuation of project.
- ◆ Participants
 - Top management, project leaders, process owners, and analysts.
- ◆ Procedure
 - Check status of documents (evaluation results).
 - Review outcome of the project.
 - Authorize start of next phase.

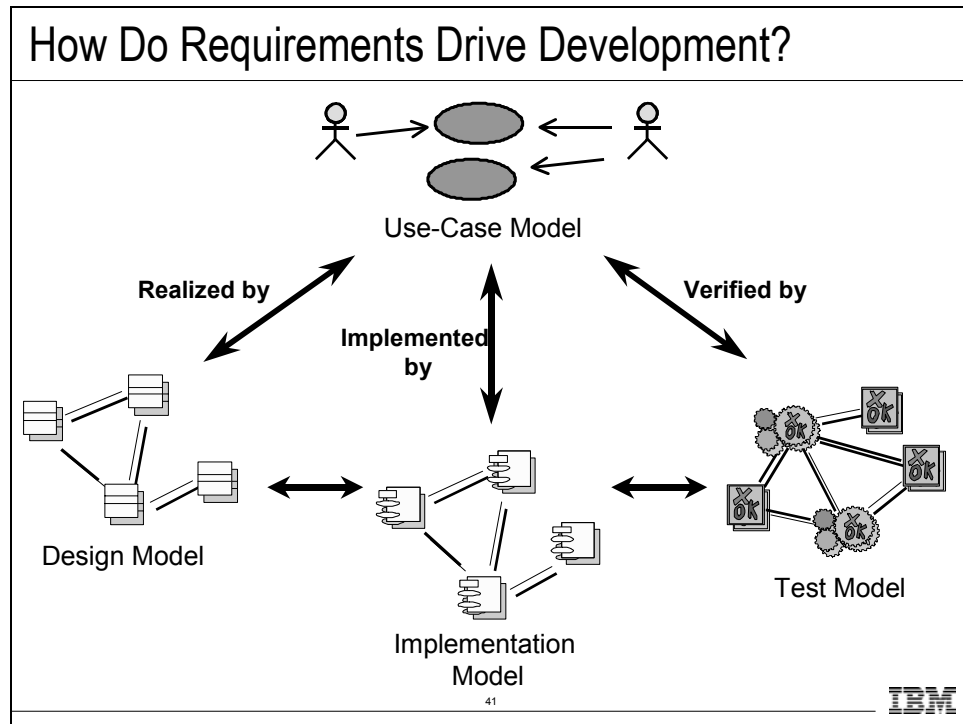
40



This is the most formal type of review.

This is usually done at formal checkpoints during the project and at the final review and acceptance point for a release to the customer.

How Do Requirements Drive Development?



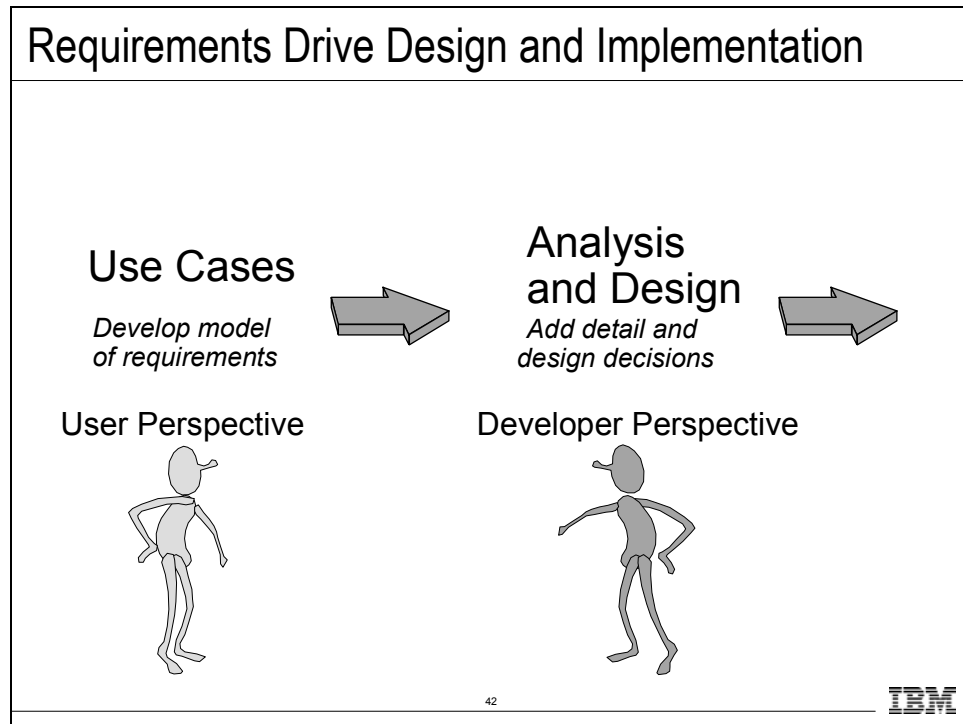
Use cases defined for a system are the basis for the entire development process. They provide the unifying thread through the system and define the behavior performed by a system. Use cases play a role in each of the core process disciplines:

- The use-case model is a result of the requirements workflow. In this early process, you need the use cases to model what the system should do from the user's point of view.
- In analysis and design, use-cases are realized in a design model. You create use-case realizations, which describe how the use-case is performed in terms of interacting objects in the design model. This model describes, in terms of design objects, the different parts of the implemented system and how the parts should interact to perform the use cases.
- During implementation the design model is the implementation specification. Because use cases are the basis for the design model, they are implemented in terms of design classes.
- During test the use cases constitute basis for identifying test cases and test procedures. The system is verified by performing each use case.

Use cases have other roles as well:

- Basis for planning an iterative development.
- Foundation for what is described in user manuals.
- Definition of ordering units. For example, a customer can get a system configured with a particular mix of use cases.

Requirements Drive Design and Implementation



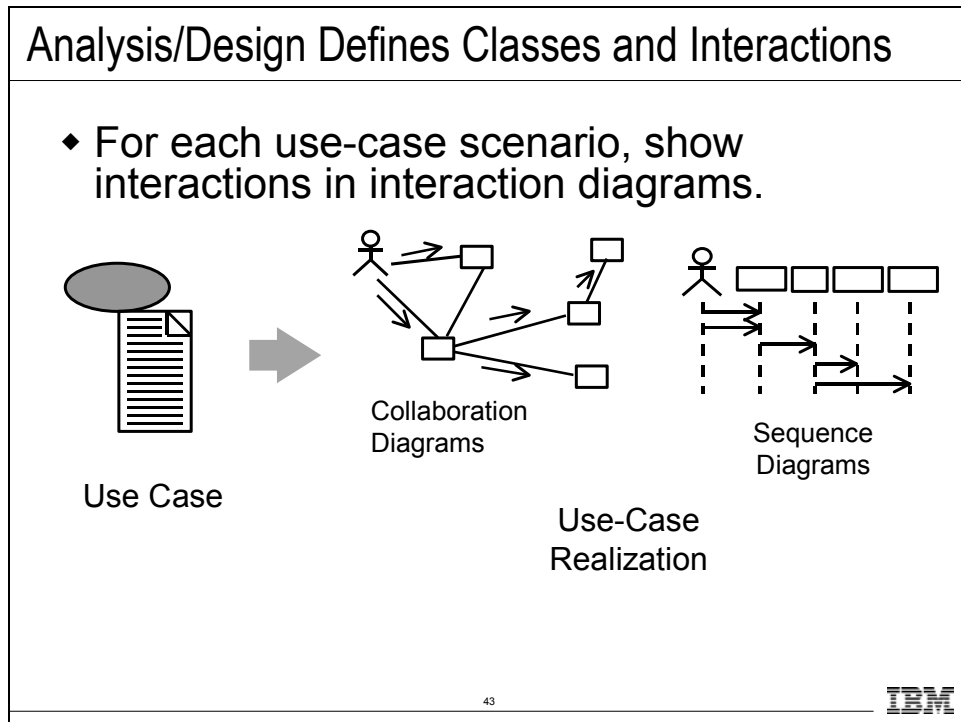
The Requirements discipline provides the primary input for Analysis and Design: Use-Case Model, Glossary, and Supplementary Specification. The Analysis and Design discipline transforms the requirements into a design of the system-to-be.

The result of analysis and design is a design model that is the "blueprint" of how the executable source code is structured and written. The design model consists of design classes structured into design packages; It also contains descriptions of how objects of these design classes collaborate to perform use cases (use-case realizations).

Use-case descriptions read like a "black-box" description, in which internal details on what the system does in response to an actor's actions is missing. To find the objects, which perform the use case, you need to have the "white-box" description of what the system does from an internal perspective.

The examples on the following pages show the kinds of activities in the analysis and design process and the kinds of diagrams that are created. Our intent is to illustrate how use cases from the Requirements discipline is used in Analysis and Design. If you want to learn more about analysis and design, Rational offers an Object-Oriented Analysis and Design (OOAD) course.

Analysis/Design Defines Classes and Interactions



You can identify classes responsible for the required behavior by stepping through the flow of events of the use case. After identifying classes, it is time to see exactly where they are applied in the use-case flow of events.

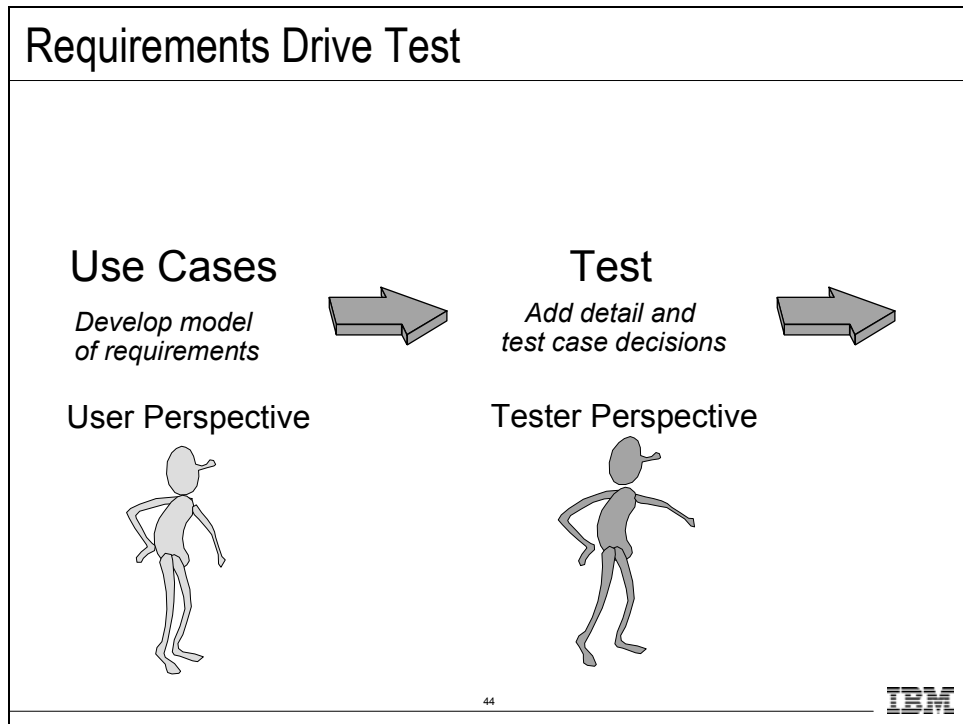
A use-case realization describes how a particular use case is realized within the design model, in terms of collaborating objects. A use-case realization ties together the use cases from the use-case model with the classes and relationships of the design model. A use-case realization specifies what classes must be built to implement each use case.

Within the UML, the analysis and design are represented using a set of diagrams which model: the classes/objects that implement the use case and their relationships, class diagrams; how these classes/objects interact to perform the use cases, collaboration and sequence diagrams.

Interaction diagrams show interactions of the system with its actors. Most of the flows of events for each use case are modeled to make sure that all requirements are handled by operations of the participating classes.

The number and types of the diagrams that are used depend on what is needed to provide a complete picture of the collaboration and the guidelines developed for the project under development.

Requirements Drive Test



What do you want to test after the design for a system is implemented into executable code? To make sure the system does what the requirements say it should do.

The requirements discipline provides the primary input for “black-box” (from the external perspective of the system) testing. Use-cases provide a “black-box” description of what the system does in response to an actor’s actions. So, a test case consists of running through the use case and seeing if the desired result is obtained. The test workflow transforms the requirements into a test case, which can then be transformed into test scripts for automated testing.

The examples on the following pages show examples of how scenarios of a use case become test cases. Our intent is to illustrate how use cases from the requirements workflow are used in test.